

CO_5 AT_3 CODING CHALLENGE

DSA1030 - C++ PROGRAMMING

NAME: Ranjitha. R

REG NO: 192421285

1. A multinational company wants to develop an Employee Performance Analytics System to evaluate employee productivity across multiple departments, where each department has a varying number of employees and performance scores. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent and manage employee data effectively. The program should compute the top performer in each department, identify the overall best employee, and calculate the average performance score per department. Additionally, the solution must handle variable department sizes, edge cases such as no employees, equal scores, or all low-performing employees, and ensure efficient comparisons, optimized time complexity, and modular program design.

1. Employee Performance Analytics System

Requirements

- Multiple departments
- Different number of employees in each department
- Store employee name, ID and performance score
- Find top performer in each department
- Find overall top performer
- Calculate average score for each department
- Handle empty departments and equal scores

C++ Program

```
#include <iostream>
```

```
#include <vector>
```

```
#include <string>
```

```
using namespace std;
```

```
class Employee {
```

```
public:
```

```
    int id;
```

```
    string name;
```

```
    double score;
```

```
    Employee(int i, string n, double s) {
```

```

        id = i;

        name = n;

        score = s;
    }
};

class Department {
public:
    string name;
    vector<Employee> employees;

    Department(string n) {
        name = n;
    }

    void addEmployee(Employee e) {
        employees.push_back(e);
    }

    void displayResult() {
        cout << "\nDepartment: " << name << endl;

        if (employees.empty()) {
            cout << "No employees in this department." << endl;
            return;
        }

        double total = 0;
        int topIndex = 0;
        for (int i = 0; i < employees.size(); i++) {
            total += employees[i].score;

            if (employees[i].score > employees[topIndex].score) {
                topIndex = i;
            }
        }
    }
};

```

```

        double average = total / employees.size();

        cout << "Top Performer: " << employees[topIndex].name
              << " (Score: " << employees[topIndex].score << ")" << endl;

        cout << "Average Score: " << average << endl;
    }
};

int main() {
    int n;

    cout << "Enter number of departments: ";

    cin >> n;

    vector<Department> departments;

    for (int i = 0; i < n; i++) {
        string deptName;

        int m;

        cout << "\nEnter department name: ";

        cin >> deptName;

        Department dept(deptName);

        cout << "Enter number of employees: ";

        cin >> m;

        for (int j = 0; j < m; j++) {
            int id;

            string name;

            double score;

            cout << "\nEnter Employee ID: ";

            cin >> id;

            cout << "Enter Employee Name: ";

            cin >> name;

            cout << "Enter Performance Score: ";

            cin >> score;

            dept.addEmployee(Employee(id, name, score));

```

```

    }

    departments.push_back(dept);
}

// Display department results
for (int i = 0; i < departments.size(); i++) {
    departments[i].displayResult();
}

// Find overall top performer
bool found = false;
Employee overallTop(0, "", 0);
for (int i = 0; i < departments.size(); i++) {
    for (int j = 0; j < departments[i].employees.size(); j++) {
        if (!found || departments[i].employees[j].score > overallTop.score) {
            overallTop = departments[i].employees[j];
            found = true;
        }
    }
}

cout << "\n-----" << endl;
if (found) {
    cout << "Overall Top Performer: "
        << overallTop.name << endl;
    cout << "Score: " << overallTop.score << endl;
} else {
    cout << "No employees available." << endl;
}

return 0;
}

```

Sample Output

```
Enter number of departments: 2

Enter department name: IT
Enter number of employees: 2

Enter Employee ID: 101
Enter Employee Name: RAM
Enter Performance Score: 97

Enter Employee ID: 102
Enter Employee Name: PRIYA
Enter Performance Score: 96

Enter department name: HR
Enter number of employees: 3

Enter Employee ID: 201
Enter Employee Name: RAMYA
Enter Performance Score: 88

Enter Employee ID: 202
Enter Employee Name: RAVI
Enter Performance Score: 92

Enter Employee ID: 203
Enter Employee Name: MOHAN
Enter Performance Score: 91

Department: IT
Top Performer: RAM (Score: 97)
Average Score: 96.5

Department: HR
Top Performer: RAVI (Score: 92)
Average Score: 90.3333

-----
Overall Top Performer: RAM
Score: 97

=== Code Execution Successful ===
```

Time Complexity

- Finding department top performer and average: $O(n)$
- Finding overall top performer: $O(n)$
- Overall: $O(n)$ where n is the total number of employees.

2. A logistics company wants to develop a Fleet Management System to monitor multiple vehicles across different routes, where each route has a varying number of vehicles and trip records. Design a C++ program using functions and object-oriented concepts such as classes, objects, access specifiers, member functions, static members, and arrays of objects to represent vehicle data efficiently. The program should compute the highest distance covered vehicle on each route, identify the overall most efficient vehicle, and calculate the average distance per route. Additionally, the solution must handle variable fleet sizes, edge cases such as no vehicles, identical distances, or zero trips, and ensure efficient data processing, modular design, and optimized comparisons..

2. Fleet Management System

Requirements

- Multiple routes
- Different number of vehicles on each route
- Store vehicle ID, driver name and distance
- Find highest-distance vehicle on each route
- Find overall most efficient vehicle
- Calculate average distance per route
- Handle empty routes, zero trips and identical distances

C++ Program

```
#include <iostream>
#include <vector>
#include <string>
using namespace std;
class Vehicle {
public:
    int id;
    string driver;
    double distance;
    Vehicle(int i, string d, double dis) {
        id = i;
        driver = d;
        distance = dis;
    }
};
class Route {
public:
    string routeName;
    vector<Vehicle> vehicles;
    Route(string r) {
        routeName = r;
```

```

    }

    void addVehicle(Vehicle v) {
        vehicles.push_back(v);
    }

    void displayResult() {
        cout << "\nRoute: " << routeName << endl;
        if (vehicles.empty()) {
            cout << "No vehicles on this route." << endl;
            return;
        }
        double total = 0;
        int maxIndex = 0;
        for (int i = 0; i < vehicles.size(); i++) {
            total += vehicles[i].distance;
            if (vehicles[i].distance > vehicles[maxIndex].distance) {
                maxIndex = i;
            }
        }
        double average = total / vehicles.size();
        cout << "Highest Distance Vehicle: Vehicle "
            << vehicles[maxIndex].id << endl;
        cout << "Driver: " << vehicles[maxIndex].driver << endl;
        cout << "Distance: " << vehicles[maxIndex].distance
            << " km" << endl;
        cout << "Average Distance: " << average
            << " km" << endl;
    }
};

int main() {
    int n;

```

```

cout << "Enter number of routes: ";
cin >> n;
vector<Route> routes;
for (int i = 0; i < n; i++) {
    string routeName;
    int m;
    cout << "\nEnter route name: ";
    cin >> routeName;
    Route route(routeName);
    cout << "Enter number of vehicles: ";
    cin >> m;
    for (int j = 0; j < m; j++) {
        int id;
        string driver;
        double distance;
        cout << "\nEnter Vehicle ID: ";
        cin >> id;
        cout << "Enter Driver Name: ";
        cin >> driver;
        cout << "Enter Distance Covered (km): ";
        cin >> distance;
        route.addVehicle(Vehicle(id, driver, distance));
    }
    routes.push_back(route);
}
// Display route results
for (int i = 0; i < routes.size(); i++) {
    routes[i].displayResult();
}
// Find overall most efficient vehicle

```



```

bool found = false;
Vehicle bestVehicle(0, "", 0);
for (int i = 0; i < routes.size(); i++) {
    for (int j = 0; j < routes[i].vehicles.size(); j++) {
        if (!found ||
            routes[i].vehicles[j].distance > bestVehicle.distance) {
            bestVehicle = routes[i].vehicles[j];
            found = true;
        }
    }
}
cout << "\n-----" << endl;
if (found) {
    cout << "Overall Most Efficient Vehicle: Vehicle "
        << bestVehicle.id << endl;
    cout << "Driver: " << bestVehicle.driver << endl;
    cout << "Distance: " << bestVehicle.distance
        << " km" << endl;
} else {
    cout << "No vehicle records available." << endl;
}
return 0;
}

```

Sample Output

```
Output
Enter number of routes: 2
Enter route name: CHENNAI
Enter number of vehicles: 2
Enter Vehicle ID: 101
Enter Driver Name: RAM
Enter Distance Covered (km): 450
Enter Vehicle ID: 102
Enter Driver Name: RAMYA
Enter Distance Covered (km): 560
Enter route name: BENGALORE
Enter number of vehicles: 1
Enter Vehicle ID: 406
Enter Driver Name: RANJITHA
Enter Distance Covered (km): 206
Route: CHENNAI
Highest Distance Vehicle: Vehicle 102
Driver: RAMYA
Distance: 560 km
Average Distance: 505 km
Route: BENGALORE
Highest Distance Vehicle: Vehicle 406
Driver: RANJITHA
Distance: 206 km
Average Distance: 206 km
-----
Overall Most Efficient Vehicle: Vehicle 102
Driver: RAMYA
Distance: 560 km
=== Code Execution Successful ===
```

Time Complexity

- Finding highest distance and average for each route: **$O(n)$**
- Finding overall vehicle: **$O(n)$**
- Overall: **$O(n)$** where n is the total number of vehicles.